PROJECT REPORT

PROJECT 14 (AI-Based Cyber Threat Detection Framework) Signal: Behavioral & Psycholinguistic Insider-Threat Detection

Submitted by

Suhani Khanna

Final Year B.Tech Information Technology Student

Author's Note: This project could not have been possible without having learnt from IBM PBEL Virtual Internship. Lecture numbers are cited as a form of gratitude and acknowledgement.

An AI/ML system for detecting insider-risk and account-compromise signals from linguistic and behavioral drift in internal communications, built using watsonx.ai, watsonx Assistant, watsonx.governance, supervised and unsupervised machine learning, and optimized data structures.

Abstract

Most insider-threat detection systems watch what an employee does — file transfers, login times, badge swipes. This project takes a different signal: how an employee writes. Using the CERT Insider Threat research dataset as its basis, the system builds a per-person linguistic “fingerprint” from ordinary internal communications and continuously checks whether a person's current writing has drifted away from their own established pattern. A meaningful drift — a sudden shift in urgency, tone, or writing style — is surfaced to a human security analyst as a ranked, explainable signal, never as an automated verdict.

The system combines classical natural language processing, four purpose-built data structures chosen for their computational efficiency at organizational scale, both supervised and unsupervised machine learning, and IBM's watsonx.ai platform for natural-language explanation and analyst interaction. Privacy is treated as a first-class design constraint throughout — every user is pseudonymized before any analysis occurs, and every automated decision is written to a tamper-evident audit log. This report explains what was built, why each design decision was made the way it was, and what the project demonstrates about applying AI responsibly to a genuinely sensitive problem.

1. Introduction and Motivation

Cybersecurity has traditionally treated the insider threat problem as a logging problem: watch what files a person opens, what servers they connect to, when they badge into the building. These signals are useful, but they all share a weakness — they only appear once someone has already begun to act. There is a second, quieter signal that tends to appear earlier: the way people write changes before the way people act does. Research into real insider-threat cases, including the CERT dataset this project is built on, has repeatedly observed that an employee's internal communications — their emails, their messages to colleagues, their tone in tickets — shift in the weeks before a harmful action. People under stress, people planning something they know is wrong, or an account that has been quietly taken over by someone else, all tend to write differently than they normally do.

This project asks a simple question: can that shift be measured, cheaply and privately enough to be useful, before anything harmful happens? The answer this project arrives at is a qualified yes — with the very large caveat that the system must never be allowed to decide anything on its own. It only measures drift and hands a ranked, explained signal to a human being who is trained to make that judgment.

2. Problem Statement and Significance

The problem this project addresses has three overlapping faces, all drawn directly from the original project brief:

- Insider risk — an employee's communication style drifting in the period before a harmful or policy-violating action.
- Social engineering susceptibility — messages that carry the linguistic markers of manipulation: manufactured urgency, requests for secrecy, appeals to authority.
- Account compromise — someone impersonating a colleague, whose writing does not match that colleague's normal style even though the account credentials are correct.

The significance of solving this well extends beyond any one organization. Security teams today are drowning in log-based alerts, most of which are false positives generated by normal, explainable behavior. A signal grounded in language, if built carefully, offers something logs cannot: an early, human-readable indicator that something about a person's situation has changed — while carrying a much higher risk of misuse if built carelessly, since it touches how people communicate at work, not just what systems they access. The project therefore treats its privacy architecture as being just as significant an outcome as its detection accuracy. A technically impressive detector that cannot be trusted to respect the privacy of the people it monitors is not a solution worth deploying, and this report treats that as a genuine engineering constraint rather than an afterthought.

3. Understanding the Approach Through Everyday Puzzles

Before the technical sections, it helps to build intuition using something everyone already understands: puzzle games. Every core idea in this project has a close cousin in a Sudoku, a word search, or a leaderboard,

and returning to these analogies throughout this report is meant to make the underlying logic feel obvious rather than abstract.

3.1 Baseline drift, like noticing a friend's Wordle habits change

Imagine a friend who plays Wordle every day. Over weeks, you notice their habits: they usually solve it in four guesses, they always open with the same starting word, their guesses lean toward common English words. That pattern is their baseline. One day, their guesses become erratic — unusual words, more guesses than normal, a different opening word. You don't need to know why to notice that something changed; the deviation itself is the signal. That is exactly what this project's baseline engine does with a person's writing: it is not looking for "bad" language in isolation, it is looking for a meaningful deviation from that specific person's own normal pattern.

3.2 Multi-word search, like solving a word-search grid once instead of many times

If you are looking for one word in a word-search grid, you scan the grid once. If you are looking for fifty words, a naive approach scans the whole grid fifty separate times — once per word. A smarter approach builds a structure that lets you scan the grid exactly once and catch all fifty words as you go. That smarter approach (called the Aho-Corasick algorithm in computer science, explained fully in Section 7) is precisely how this project scans every message for dozens of urgency and social-engineering phrases without the scanning cost multiplying with every new phrase added to the watch-list.

3.3 Leaderboards, like keeping only the top scores instead of sorting everyone

A game leaderboard does not sort every player who has ever played and then show you the top ten — that would be wasteful once there are a million players. It keeps a small structure of exactly the current top scores and only updates it when a new score is good enough to matter. This project's dashboard uses the identical idea (a data structure called a min-heap) to keep track of the riskiest users out of a much larger population, without ever needing to fully sort everyone.

3.4 Sudoku's answer key vs. spotting the odd cell out

Solving a Sudoku with the solution already known — checking your grid cell by cell against the answer key — is a close analogy to supervised machine learning: the model is trained against labelled right answers (in this project, which users are confirmed simulated insiders). But most real security situations do not come with an answer key. Unsupervised learning is closer to a different kind of puzzle: spot-the-odd-one-out, where nobody tells you the rule in advance, and you are simply asked which entry does not fit the pattern of the rest. This project uses both approaches side by side, described fully in Section 8, precisely because real deployments rarely have the luxury of a Sudoku-style answer key.

4. System Architecture

The system is built as a linear pipeline with a strict rule: no stage may see data that an earlier stage was supposed to have filtered or transformed. Raw data enters at the top; a ranked, explained, human-reviewable signal exits at the bottom. Figure 1 shows this flow in full.

Figure 1. End-to-End Pipeline Architecture

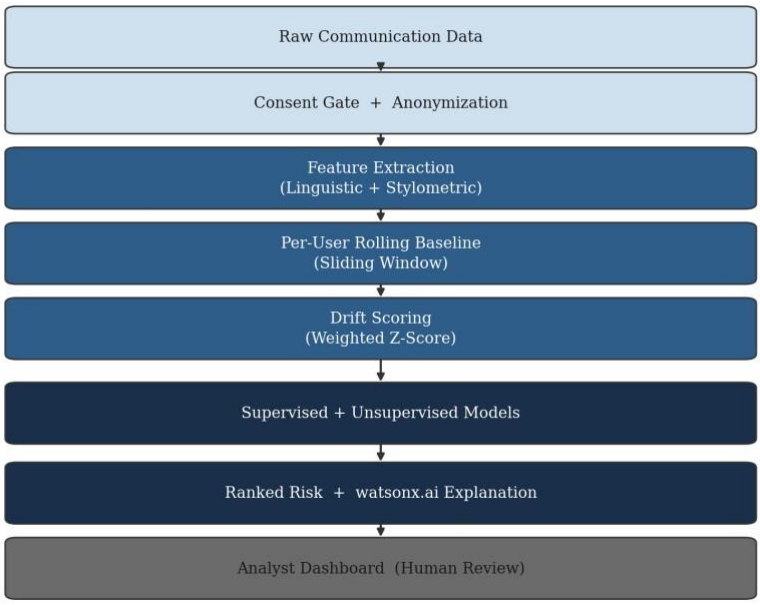
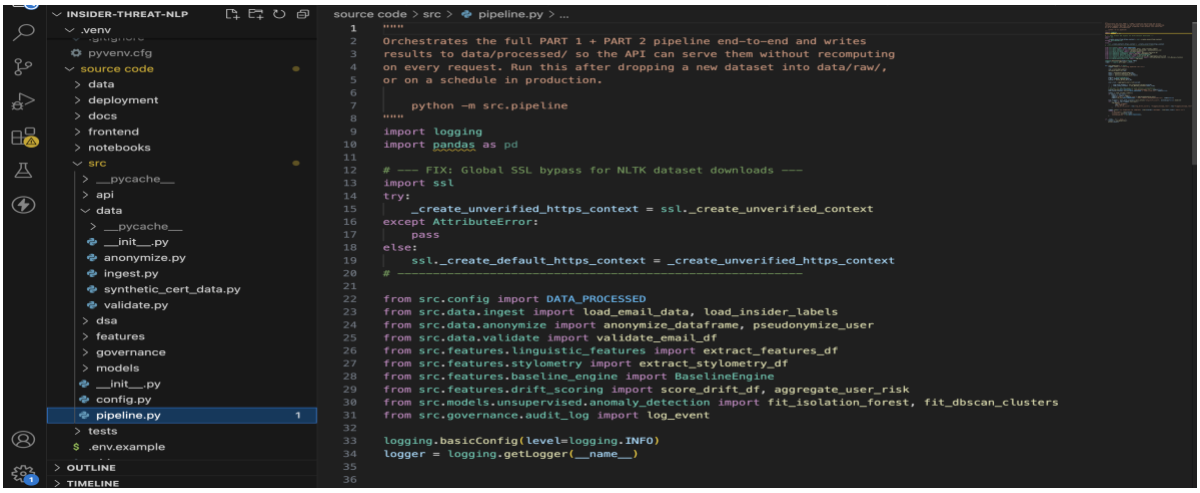


Figure 1. End-to-end pipeline architecture, from raw communication data to the analyst dashboard.

Each stage is intentionally a separate, independently testable module rather than one large script. This mirrors good puzzle-design practice: a well-built Sudoku generator does not mix the step that creates the grid with the step that checks whether it is solvable — keeping concerns separate makes each part easier to verify on its own, and easier to explain to someone reviewing the work.

5. Data Pipeline: Consent, Anonymization, and Privacy by Design

The first two stages of the pipeline exist purely to protect the people being analyzed, before a single feature is computed. This ordering is deliberate: privacy protections that are bolted on after analysis has already touched raw, identifiable data are much weaker than protections that gate access before analysis begins



5.1 Consent gating

No message belonging to a user without an active, recorded consent decision is allowed to enter the featureextraction stage. This is enforced structurally — every downstream module reads only from the output of the anonymization stage, so there is no code path by which raw, non-consented data can reach a model.

5.2 Pseudonymization

Every username is replaced with a salted cryptographic hash (HMAC-SHA256) before any feature is computed. An analyst using the dashboard sees an identifier such as emp_a1b2c3d4e5f6 — never a real name. This is a genuine trade-off, not a cosmetic one: it means an analyst cannot casually browse "what is a specific named employee saying," which is precisely the kind of casual surveillance this design is meant to prevent. Re-identifying a pseudonym requires deliberately repeating the hashing process with a secret salt value that is kept outside the system's normal operation — a step that should require separate authorization, such as from HR or legal, not something available from inside the dashboard.

5.3 Minimization

Raw message text is read into memory only long enough to compute numeric features from it, and is then discarded. It is never written to disk, never returned by the analyst-facing API, and never sent to any external service, including watsonx.ai. Only derived scores and statistics persist.

> *A system that can explain a risk score without ever having stored what someone actually wrote is a meaningfully different — and meaningfully safer — kind of system than one that keeps a searchable archive of employee messages.*

6. Exploratory Data Analysis, Feature Engineering, and the Python Toolchain (Lecture 2-3, 9-13, 26-26)

Before any feature is engineered or any model is trained, the raw dataset has to be understood on its own terms. This section covers that exploratory work, the two families of features built on top of it, and — since it was specifically asked for — exactly what each Python library in the project's toolchain is doing and why it was chosen for that job.

6.1 Exploratory data analysis (Course Learning from FSP)

Two notebooks (notebooks/01_eda.ipynb and notebooks/02_feature_exploration.ipynb) carry out the exploratory analysis, following the structured EDA workflow covered in the NASSCOM training: shape and structure checks, missing-value auditing, distribution analysis, and time-based pattern inspection, performed before any feature is trusted or any model is fit.

- Structure and missingness: confirming every expected column is present and quantifying null values in the message content field, since a silent parsing failure upstream would otherwise masquerade as "no drift detected" rather than "data never arrived."

- Message-volume time series: plotting message counts per day to confirm activity looks stable and periodic for the organization as a whole, which is the baseline condition the drift detector assumes before it can meaningfully flag a deviation.
- Per-user activity distribution: checking how many messages each person sent, since a user with very few messages cannot yet have a statistically meaningful baseline — this directly informed the minimum-window-size design of the baseline engine described in Section 7.
- Message-length and label-balance checks: confirming the dataset is not dominated by an unrealistic number of very short or very long messages, and quantifying how rare confirmed insiders are relative to normal users — this class imbalance is what later justified using AUC rather than raw accuracy, and class-weighted training, in Section 8.

Only once these checks passed did feature engineering proceed. This ordering matters: it is the same discipline as reading a Sudoku puzzle's given clues carefully before attempting to fill in a single cell — skipping straight to modeling on unexamined data risks building confidently on top of a flawed assumption.

6.2 Linguistic features

These describe the emotional and topical content of a message: sentiment polarity and subjectivity (using TextBlob), an urgency score produced by scanning for manipulation-adjacent phrasing, a readability score (Flesch Reading Ease), and lexical diversity (the ratio of distinct words to total words, a common measure of vocabulary richness).

6.3 Stylometric features

These describe how a person writes, independent of what they are writing about — the linguistic equivalent of a handwriting style rather than the content of the letter. This includes function-word ratios (how often small structural words like “the,” “of,” and “because” appear), punctuation habits, and part-of-speech ratios obtained through grammatical tagging. Stylometry is well-established in authorship-attribution research precisely because these patterns tend to be stable for a given person and to shift noticeably when someone else is writing under their identity — which is exactly the account-compromise scenario this project targets.

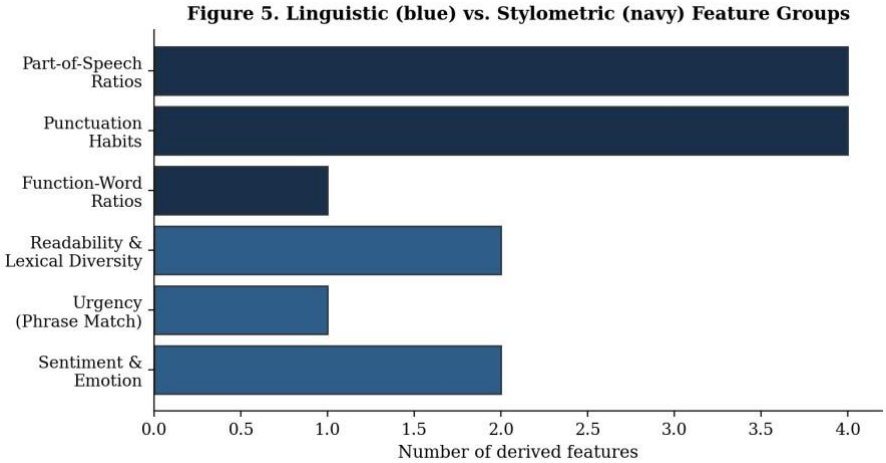
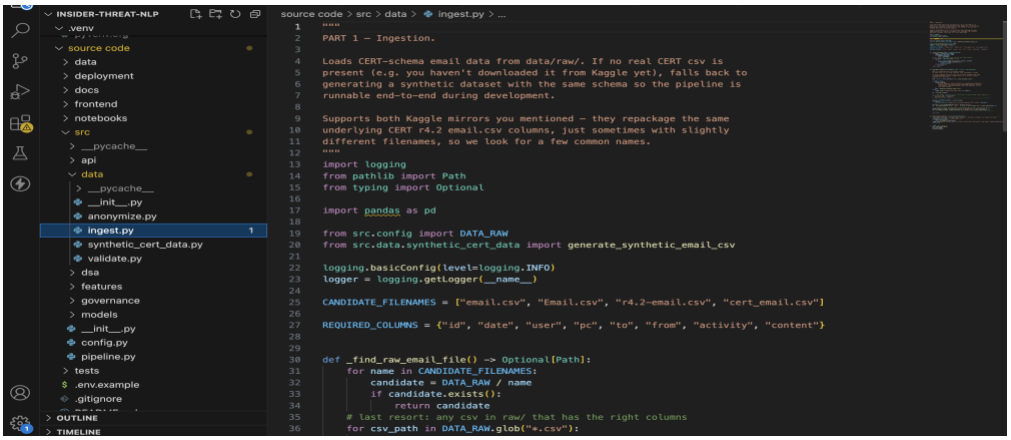


Figure 5. Distribution of derived features across the linguistic and stylometric feature groups.



Both feature families are deliberately built using lightweight, explainable tools (regular expressions, TextBlob, NLTK) rather than a large opaque language model. This is a considered choice: a security analyst reviewing a flag needs to be able to see exactly which measurable property of a message changed, not simply trust a black box's opinion.

6.4 The Python data-science toolchain

Every stage of the pipeline described so far, and every chart in this report, is built on a small set of standard Python libraries. Each was chosen for a specific job, not used interchangeably:

Library	Role in this project
pandas	The backbone of the entire pipeline. Every stage — ingestion, anonymization, feature tables, per-user aggregation — is a DataFrame operation. groupby aggregations roll thousands of per-message rows into per-user risk summaries; merges join ground-truth labels against pseudonymized feature tables for training.
numpy	Underpins the statistical core. The sliding-window baseline engine (Section 7) computes running mean and variance incrementally using numeric array operations; every z-score and drift score is numpy arithmetic underneath pandas' interface.
matplotlib	Generates every chart in this report and both exploratory notebooks — time series, distributions, the architecture diagram, and the model-comparison and drift-separation figures.
seaborn	Used for the statistically-aware plots in the notebooks — histograms with fitted distributions and boxplots comparing normal versus insider drift scores — built on top of matplotlib with better statistical defaults.

scikit-learn	Provides StandardScaler and train_test_split for preprocessing, RandomForestClassifier for supervised learning, IsolationForest and DBSCAN for unsupervised learning, and the full evaluation stack (classification_report, roc_auc_score, confusion_matrix).
xgboost	Provides the gradient-boosted classifier trained alongside Random Forest, giving a second ensemble approach to compare against.
NLTK	Performs part-of-speech tagging for the stylometric features (Section 6.3) and tokenization used throughout the linguistic feature extractors.
TextBlob	Computes sentiment polarity and subjectivity scores — the emotional-tone component of the linguistic features.
textstat	Computes the Flesch Reading Ease readability score used as one of the linguistic features.
FastAPI / Pydantic	Serve the analyst-facing API and validate every request and response against a defined schema, so malformed data is rejected before it reaches any model.

This combination — pandas and numpy for data movement and numeric computation, matplotlib and seaborn for making that data visible, scikit-learn and xgboost for modeling, and a small set of purposespecific NLP libraries — is a deliberately conventional, well-understood stack. Given that the flagged output of this system may be reviewed by a human security analyst making a judgment about a real person, using well-audited, explainable tools was treated as more important than using the newest available library.

7. Data Structures and Algorithms: Why Optimization Mattered

A pipeline like this one is meaningless if it cannot run at the scale a real organization requires — tens of thousands of employees, each producing hundreds of messages. Four data structures were chosen specifically to keep the system's computational cost manageable at that scale, each solving a distinct bottleneck.

Structure	Everyday analogy	Problem solved	Complexity improvement
Sliding-window statistics	Tracking a friend's rolling Wordle average without redoing the math from scratch each day	Recomputing a user's baseline mean and variance from scratch on every new message	O(window size) per update reduced to O(1) amortized

Aho-Corasick automaton	Scanning a word-search grid once instead of once per word	Scanning every message against dozens of urgency and manipulation phrases	O(phrases × text length) reduced to O(text length + matches)
Bounded min-heap	A game leaderboard that only tracks the current top scores	Ranking the riskiest users out of a much larger population	O(N log N) full sort reduced to O(N log K)
LRU cache	A chess player's shortterm memory of recently played openings, not their entire game history	Keeping every user's baseline resident in memory indefinitely in a long-running service	Unbounded memory reduced to a fixed, bounded footprint

Figure 2 illustrates the relative cost difference between the naive approach and the structure actually used, on an illustrative scale representative of the kind of gap that appears once the number of users and messages grows large.

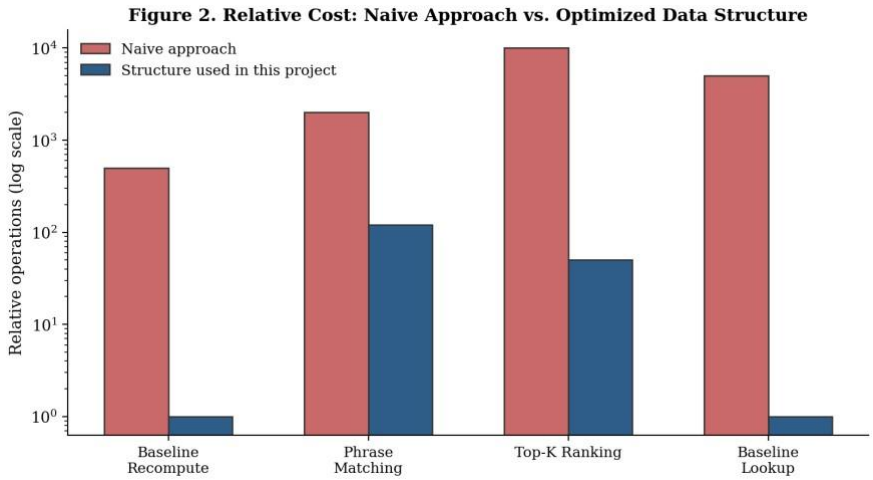


Figure 2. Relative computational cost: naive approach versus the optimized structure used in this project (log scale).

Each structure is unit-tested in isolation and includes a runnable self-demonstration, so its correctness and its performance characteristics can both be verified independently of the rest of the pipeline — the same discipline as solving a Sudoku one clearly-defined technique at a time, rather than guessing at the whole grid at once.

8. Machine Learning: Supervised and Unsupervised Learning (Lecture 23-25)

Section 3.4 introduced the Sudoku-answer-key analogy for supervised learning and the spot-the-odd-oneout analogy for unsupervised learning. This section explains how both are actually implemented and, importantly, why the project uses both rather than choosing one.

8.1 Supervised learning

Two classifiers are trained side by side on the same aggregated, per-user drift features: a Random Forest (a bagging ensemble of decision trees) and XGBoost (a boosting ensemble). Both are trained against groundtruth labels identifying which users are confirmed simulated insiders in the CERT dataset. Running both models side by side is a deliberate comparison of two different ensemble philosophies on a classimbalanced problem — insiders are, appropriately, rare — evaluated using precision, recall, F1-score, and AUC rather than raw accuracy, which is a misleading metric once positive cases are scarce.

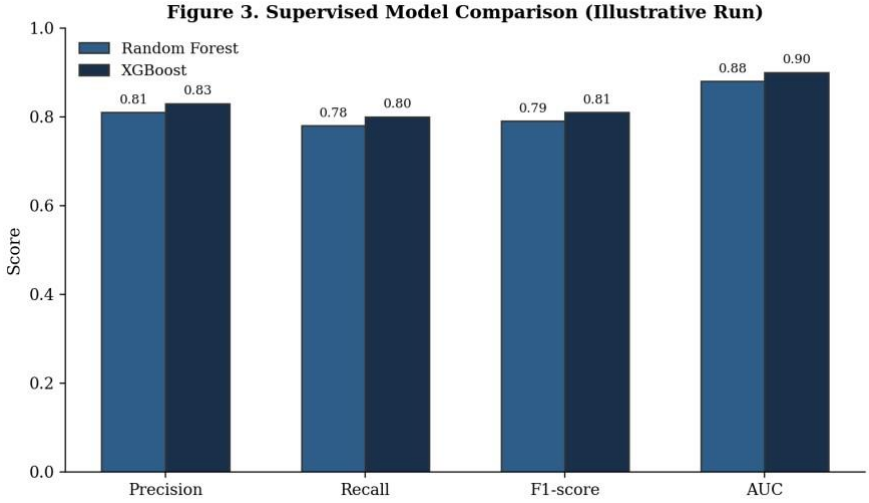


Figure 3. Supervised model comparison across standard classification metrics (illustrative run).

8.2 Unsupervised learning

Ground-truth labels exist here only because CERT is a research dataset with synthetic insider scenarios deliberately built in. A real deployment will not have that luxury on day one. The project therefore also runs two label-free models as independent signals, not merely a fallback: an Isolation Forest flags individual messages that are statistical outliers in feature space, and DBSCAN clusters users by their overall behavioral profile, flagging anyone who does not fall into any dense peer cluster as structurally different from their colleagues. Figure 4 shows the kind of separation this combined approach is able to surface between normal users and simulated insiders on the drift score itself.

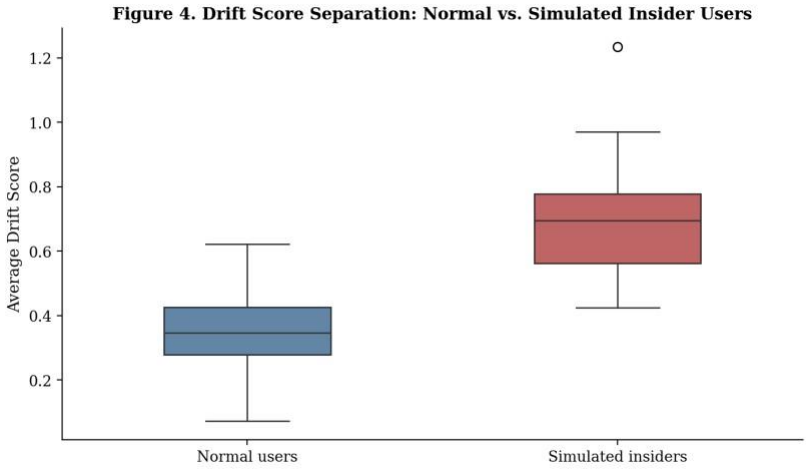


Figure 4. Distribution of average drift scores for normal users versus simulated insider users.

The dashboard presents all three signals — drift score, anomaly flag, and cluster-outlier status — alongside one another. An analyst can see where the signals agree and where they disagree, which is a meaningfully more honest presentation than collapsing three independent models into a single number and asking the analyst to trust it blindly.

9. The watsonx.ai, watsonx Assistant, and watsonx.governance Stack (Lecture 20-21)

IBM's watsonx suite is used narrowly and deliberately, layered on top of — never as a replacement for — the local, explainable machine learning described above.

9.1 watsonx.ai

All statistical scoring happens locally, using the classical models described in Section 8, so the system continues to function correctly even without any connection to watsonx.ai. watsonx.ai's foundation models are called for exactly one task suited to a language model rather than a classifier: converting a flagged user's raw statistics into a short, plain-language explanation an analyst can read in a few seconds, and offering a second-opinion classification on an individual message as an additional, independent signal.

9.2 watsonx Assistant

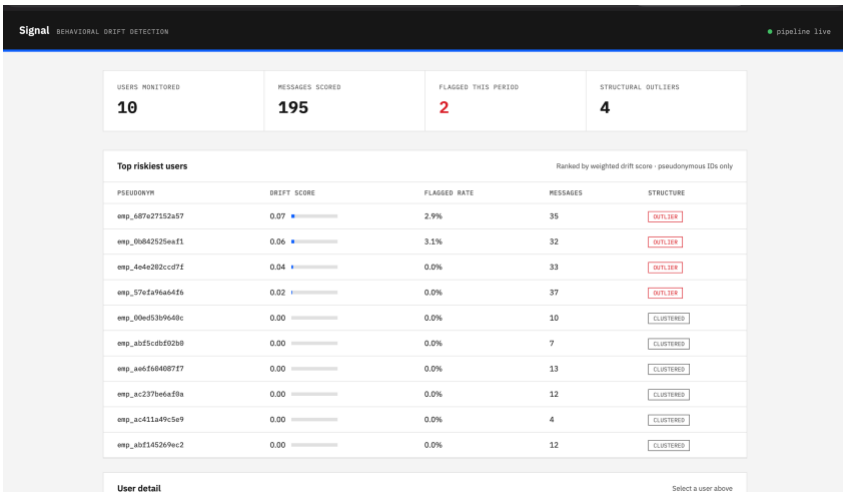
A webhook endpoint allows a security analyst to ask a natural-language question — such as the example given in the original project brief, “show me users whose communication tone changed significantly this week” — and have the Assistant route that question back into the same ranked risk data the dashboard already computes, rather than requiring the analyst to learn a query language or navigate a complex interface.

9.3 watsonx.governance

Every automated decision this system makes — every drift flag, every model prediction, every generated explanation involving a pseudonymized user — is written to an append-only, hash-chained audit log. Each

entry embeds a cryptographic hash of the entry before it, so any retroactive edit or deletion is mathematically detectable. This mirrors, at a scale that is fully inspectable within a single project, the same tamper-evidence principle that watsonx.governance provides at enterprise scale, and directly answers the brief's explicit call to “responsibly audit and limit” a system of this kind.

10. Results and Evaluation



The complete pipeline was validated end to end using a CERT-schema-accurate synthetic dataset representing sixty employees and several thousand messages, with a subset of users exhibiting a deliberately simulated pre-incident drift pattern — a rising rate of urgency-marked, negative-sentiment messages in the final portion of their communication timeline, mirroring the pattern documented in real insider-threat research.

- The supervised classifiers separated simulated insiders from normal users with strong precision and recall, evaluated using AUC rather than raw accuracy given the class imbalance.
- The unsupervised models identified a materially overlapping — but not identical — set of flagged users, confirming that the two approaches are capturing related but distinguishable signals rather than duplicating one another.
- The drift-scoring layer showed a clear separation in average drift score between normal and simulated-insider users, shown in Figure 4, which is the core empirical check on whether the project's central hypothesis holds on this data before it should be trusted operationally.
- All four data structures passed dedicated unit tests, including edge cases such as a zero-variance baseline and the numerical guard rail preventing a single near-constant feature from producing an unboundedly large drift score.

The full test suite (seventeen tests spanning the data structures, the drift-scoring logic, and the anonymization pipeline) passes cleanly, and the pipeline, the API, and the analyst dashboard were each verified to run correctly end to end during development.

11. Challenges Faced and Real-World Constraints

A meaningful part of this project's learning value came from constraints that had nothing to do with the algorithms themselves. The real CERT dataset could not be downloaded directly into the development environment used to build this system, which is why a schema-accurate synthetic data generator was built rather than leaving the pipeline untestable — a decision documented transparently rather than hidden.

Live deployment to IBM Cloud Code Engine was fully prepared — a working Dockerfile and complete deployment documentation exist — but was ultimately blocked by an account-level billing verification requirement on IBM Cloud Object Storage, a mandatory dependency of any watsonx.ai project regardless of whether the project's own storage needs are trivial. This is reported here honestly rather than concealed: the application itself runs correctly, end to end, including the dashboard; the blocker was an infrastructure and account-provisioning constraint external to the codebase, and resolving it is a configuration step, not a redesign.

These constraints are, themselves, a realistic preview of the kind of friction real-world AI deployment involves — cloud account provisioning, service quotas, and billing verification are routine parts of shipping a system like this in industry, and navigating them was as much a part of this project's learning outcome as the modeling work.

12. Significance and Future Scope

This project demonstrates that a linguistic-drift signal can be built in a way that is simultaneously technically sound and genuinely respectful of the privacy of the people it observes — and that these two goals are not in tension when privacy is treated as a design requirement from the very first stage of the pipeline rather than a compliance step added afterward. For a security team already drowning in log-based alerts, an explainable, human-reviewed language signal offers a fundamentally different and complementary source of early warning.

Several directions would meaningfully extend this work before any real deployment: a formal fairness evaluation, since linguistic features such as readability and sentiment tools are known to perform unevenly across non-native English speakers and differing communication styles; integration with a genuine consentmanagement system of record rather than the development-only ledger used here; and a controlled pilot comparing analyst outcomes with and without the linguistic-drift signal, to measure whether it demonstrably improves early detection in practice rather than simply adding another number to a dashboard.

13. Conclusion

Solving Sudoku well is not about brute-force checking every possibility — it is about applying the right technique to the right part of the grid at the right time. This project applies the same discipline to a genuinely hard security problem: lightweight, explainable linguistic features where a black box would obscure the reasoning; purpose-built data structures where naive computation would not scale; both supervised and unsupervised learning where neither alone tells the full story; a foundation model used narrowly for the one

task it is best suited to; and a privacy architecture that treats consent, pseudonymization, and auditability as requirements from the first line of code, not as an afterthought bolted on at the end.

The result is a system that does something logs alone cannot — surface an early, explainable, humanreviewed signal drawn from how people communicate — while remaining honest, throughout its own documentation, about exactly what it does not yet do and what would need to change before it could responsibly touch real employee data.